

Prompting BP

Cursor AI 실습 회고 & 현업 적용

VSCode C++ · Java · Python 개발자 과정

 C++17

 Java 21

 Python 3.12

1 실습 회고

KPT 구조화 회고 + 개인 학습 성과

2 Prompting BP 사례

C++ · Java · Python 실전 우수 프롬프트 패턴

3 팀 액션 아이템

SMART 목표 + 현업 적용 계획

4 현업 적용 로드맵

1개월 · 3개월 · 6개월 단계별 계획

섹션 1

실습 회고 KPT 워크숍

Keep · Problem · Try 구조화 회고

 C++ · Java · Python 실습에서 잘 된 것들 — 각자 포스트잇 작성 → 공유

코드 탐색

Java

@Codebase로 TDD_TV 전체 구조를 10분 만에 파악 → 기존 2~3시간 대비 획기적 단축

C++

@Codebase로 ITuner 의존성 그래프 자동 생성 → 구조 이해 가속

Python

@Codebase로 FastAPI 라우터 맵 파악 → 엔드포인트 30개 10분 완료

테스트 자동화

Java

/test로 JUnit 5 경계값 테스트 자동 생성 → 커버리지 40%→80%

C++

@FakeTuner.h 참조해 Google Test 자동 생성 → 반복 작업 90% 감소

Python

@main.py /test로 pytest 픽스처 자동 생성 → 커버리지 35%→82%

리팩토링 제안

Java

Composer(Ctrl+I)로 SRP 위반 TVController 분리 → 5분 만에 완성

C++

Cmd+K로 God Class 분리 → AgedBrieltItem-SulfurasItem 즉시 적용

Python

Cmd+K로 단일 함수 분리 → 모듈별 책임 명확화, 코드 스멜 50% 제거

문서화·보고서

Java

review_report.md 자동 생성 → IMRAD 보고서 5분 완성

C++

Doxygen 주석 일괄 추가 → JSDoc 적용 시간 95% 절감

Python

Sphinx docstring 자동 완성 → API 문서 배포 시간 80% 단축

 Keep 항목은 팀 .cursorrules에 Best Practice로 등록하세요. 다음 프로젝트에서도 자동 적용됩니다.

Problem — 무엇이 팀을 느리게 했는가?

🔍 할루시네이션 & 검증 부담

- Java: 없는 Spring Boot API 생성 → 런타임 에러, 매번 실행 검증 필요
- C++: 없는 STL 메서드 참조 → 컴파일 에러, C++14/17 혼동
- Python: 폐기된 라이브러리 import 제안 → pip 설치 후 ImportError 발생

📄 컨텍스트 손실

- Java: 긴 대화 후 초기 요구사항 망각 → 엉뚱한 클래스 수정
- C++: 대용량 헤더 참조 시 컨텍스트 초과 → 분할 참조 전략 필요
- Python: notebook 환경에서 셀 순서 혼동 → 잘못된 변수 상태 참조

🔥 프롬프트 엔지니어링 부족

- Java/C++/Python: "리뷰해줘" 수준의 모호한 프롬프트 → 표면 분석만
- Python: 타입 힌트·asyncio 버전 미명시 → 호환 불가 코드 생성

🔄 Diff 검토 생략

- Java: Composer Accept All 남발 → 의도치 않은 파일 수정
- Python: requirements.txt 자동 수정으로 버전 충돌 발생

Try — 다음에 다르게 해볼 것들

할루시네이션

→ 생성 직후 즉시 빌드·테스트 실행을 프롬프트에 포함
"구현 후 빌드하고 에러 있으면 수정해줘" / `pytest -v` 실행 명시

컨텍스트 손실

→ @파일 명시적 참조 + 30분마다 핵심 요구사항 재주입
CLAUDE.md / cursorrules / pyproject.toml 상시 로드

모호한 프롬프트

→ P-C-T-F 4단계 + 7단계 구성법 적용
출력 형식(Markdown 표·코드 블록·타입 힌트)을 매번 명시

Diff 검토 생략

→ 'Accept All' 금지 → 파일별 개별 Diff 확인 후 수락
Python: requirements.txt 변경 사항 별도 확인 규칙화

섹션 2

Prompting BP 우수 사례

C++ · Java · Python 실전에서 검증된 Best Practice 패턴

 Before — 비효율적 프롬프트

"이 코드 리뷰해줘."
"TVController 좀 봐줘."
"뭐가 문제야?"

 After — P-C-T-F 구조화 프롬프트 (3개 언어 공통)

[P] 당신은 10년 경력 시니어 아키텍트입니다.
[C] @파일명 - TDD, Java21/C++17/Python3.12
[T] SOLID·코드스멜·테스트 가능성 관점으로 리뷰
[F] Critical/Major/Minor 분류 + Before/After 표

 아키텍처 설계 우수 사례 — 3개 언어 구조 설계 프롬프트 Java

@Codebase UnitConverter를 OCP/SRP로 설계.
조건:
- LengthUnit 인터페이스: 새 단위 추가 시
 기존 코드 수정 없음
- InputParser, ConversionEngine,
 OutputFormatter 클래스 분리
- units.json 외부 설정 로드
구현 후 mvn test 실행 후 수정.

 C++17

@Codebase C++17로 UnitConverter 설계.
조건:
- ILengthUnit 순수 가상 함수 OCP
- unique_ptr<ILengthUnit> 소유권
- nlohmann/json 외부 설정 로드
- Google Test 단위 변환 검증
CMakeLists.txt 수정도 알려줘.

 Python 3.12

@Codebase Python으로 UnitConverter 설계.
조건:
- ABC 기반 LengthUnit 추상 클래스
- dataclass + __init_subclass__ OCP
- pydantic으로 units.json 검증
- pytest + hypothesis 속성 테스트
uv run pytest 실행 후 수정.

95%

설계 초안 시간 단축

10분

PRD → 클래스 구조 완성

3배

리뷰 커버리지 향상

0건

설계 논의 중 구조 오류

⚡ 성능 최적화 우수 사례

⚙️ C++17 — BMI 성능 최적화

@BMICalculator.cpp 성능 최적화:

1. `std::vector`→`unordered_map`
연령대 그룹핑 $O(n)$ 달성
 2. `std::transform` + `lambda`
함수형 파이프라인 구성
 3. `getline` 대신 `mmap` 검토
(100만건 벤치마크 포함)
- C++17 `std::execution::par`
적용 가능성 분석해줘.

🍵 Java — SHealth 성능 분석

@SHealthAnalyzer.java 성능 분석:

1. $O(n^2)$ 루프→Stream API 최적화
 2. 나이대별 평균→`Map<Integer, Double>`
캐싱으로 중복 계산 제거
 3. 100만건 CSV: `BufferedReader`
메모리 최적화
- 각 개선 전/후 성능 차이도
설명해줘.

🐍 Python 3.12 — 데이터 최적화 (신규)

@bmi_analyzer.py 성능 최적화:

1. `list comprehension`→
`numpy vectorize` (100배+↑)
2. `groupby`→`polars LazyFrame`
(`pandas` 대비 10x 속도)
3. 100만건: `chunked read` with
`pyarrow` + 메모리 매핑
4. `asyncio` + `aiofiles` 비동기
파일 I/O 적용 가능성 분석.

📊 효과 지표 — C++ · Java · Python 공통

🕒 최적화 분석 시간

4~6시간



30~40분

📊 성능 향상률

수동 분석 한계



최대 100x

🐛 병목 발견 시점

배포 후



커밋 전

🔍 리팩토링 자신감

낮음



높음



테스트 자동화 우수 사례

C++ — Google Test

```
@TVChannelController.h /test
Google Test 케이스 작성:
[구조]
TEST_F(TVCtrlTest,[시나리오]){
    // Given: FakeTuner 초기 설정
    // When: 버튼 입력 시뮬
    // Then: EXPECT_EQ 검증
}
[필수케이스]
- 숫자버튼 채널 변경(경계값)
- 선호채널 토글 add/remove
- 다음선호채널 wrap-around
ctest --output-on-failure 실행.
```

Java — JUnit 5 + Mockito

```
@TVChannelController.java /test
JUnit 5 테스트 작성:
[커버리지 목표] 90% 이상
[우선순위]
1. 경계값: 채널 0,99,두자리
2. 상태전이: AwaitingFirst/Second
3. 예외: null입력, ITuner 예외
4. Mock: Mockito verify()로
   setCH 호출 검증
FIRST 원칙 준수 확인.
```

Python — pytest + hypothesis (신규)

```
@channel_controller.py /test
pytest 케이스 작성:
[커버리지 목표] 90%+
[우선순위]
1. 경계값: hypothesis로 자동생성
   @given(st.integers(0,99))
2. 픽스처: FakeTuner conftest
3. 비동기: pytest-asyncio 적용
4. Mock: unittest.mock.patch로
   set_channel 호출 검증
uv run pytest --cov=. 실행.
```

🕒 테스트 작성 시간

4~6시간 → 30~40분

📊 커버리지 달성률

평균 40% → 평균 85%+

🐞 회귀 버그 발견

배포 후 → 커밋 전

🔍 리팩토링 자신감

낮음 → 높음



Python의 hypothesis 라이브러리는 속성 기반 테스트로 엡지케이스를 AI 없이도 자동 탐색합니다.

 CI/CD & 배포 자동화 우수 사례 — GitHub Actions 연동 Java Maven — CI 파이프라인

GitHub Actions 워크플로우 작성:
조건:

- Java 21 + Maven
 - Push/PR 시 자동 실행
 - Build→Test→Coverage→PR코멘트
- ```
.github/workflows/ci.yml:
1. mvn compile (빌드 검증)
2. mvn test (JUnit 5)
3. JaCoCo 80% 미달 시 Fail
4. 결과를 PR 코멘트로 게시
```

 C++ CMake — CI 파이프라인

GitHub Actions C++ CI:  
조건:

- C++17 + CMake + Google Test
  - Ubuntu + Windows 멀티플랫폼
- ```
.github/workflows/ci.yml:
1. cmake -B build -DCMAKE_BUILD_TYPE=Debug
2. cmake --build build
3. ctest --output-on-failure
4. gcov/lcov 커버리지 측정
5. GitHub Pages 커버리지 배포
```

 Python — CI 파이프라인 (신규)

GitHub Actions Python CI:
조건:

- Python 3.12 + uv + pytest
 - Push/PR 자동 실행
- ```
.github/workflows/ci.yml:
1. uv sync --frozen
2. uv run ruff check . (lint)
3. uv run mypy . (type check)
4. uv run pytest --cov
5. coverage 80% 미달 Fail
6. Codecov 커버리지 리포트
```

 Python 전용 도구 스택 — 현업 적용 권장

## uv

pip/venv 대체 패키지 관리자  
(10~100x 빠름)

## ruff

flake8+isort+pyupgrade  
통합 린터 (Rust 기반)

## mypy

정적 타입 검사  
타입 힌트 강제화

## hypothesis

속성 기반 테스트  
자동 엡지케이스 생성

섹션 3

# 팀 액션 아이템 & SMART 목표

---

구체적·실행 가능한 개선 계획 수립

📅 팀 액션 아이템 추적 템플릿 (SMART 목표)

| 영역        | 구체적 액션                                              | 책임자    | 기한  | 성공 기준                  | 상태   |
|-----------|-----------------------------------------------------|--------|-----|------------------------|------|
| Prompting | .cursorrules 팀 공용 파일 작성 (P-C-T-F + Python 타입 힌트 규칙) | 팀장     | 1주  | 모든 PR에서 일관된 AI 응답      | 🟡 진행 |
| 테스트 품질    | 커버리지 80% 미달 PR Fail 규칙 CI에 추가 (Java/C++/Python)     | DevOps | 2주  | main 브랜치 80%+ 유지       | ☐ 예정 |
| 코드 리뷰     | Cursor AI 리뷰 → PR 코멘트 자동화 (3개 언어)                   | 전체     | 3주  | 수동 리뷰 시간 50% 단축        | ☐ 예정 |
| 문서화       | Javadoc/Doxygen/Sphinx 일괄 생성 스크립트 공유                | 팀원 A   | 1개월 | 전체 public API 100%     | ☐ 예정 |
| Python 적용 | FastAPI 레거시 Flask 모듈 1개 선정 → AI 리팩토링 시범             | 팀원 B   | 3개월 | 코드 스멜 50%↓ / 커버리지 70%↑ | ☐ 예정 |

🌟 개인별 학습 성과 공유 (6~7분 발표)

- 💡 새로 배운 기술

@Codebase 활용법 / C++ STL 최적화 / JUnit 5 / Python hypothesis 속성 테스트
- 📈 역량 향상

프롬프트 엔지니어링 / 코드 리뷰 관점 / TDD 사이클 속도 / Python 타입 시스템
- 📖 추천 자료

cursor.directory (Rules 공유) / Anthropic Prompt Guide / Real Python / Clean Code Korea
- 📌 발표 구조:

① 가장 임팩트 있었던 AI 활용 순간 → ② Before/After 비교 → ③ 현업 적용 계획 → ④ 팀 공유 인사이드

섹션 4

# 현업 적용 로드맵

---

1개월 · 3개월 · 6개월 단계별 AI 내재화 계획

## 현업 AI 내재화 타임라인

### 1개월

즉시 적용 — 도구 정착

- 팀 공용 .cursorrules 파일 완성 & Git 등록
- Cursor AI로 일일 코드 리뷰 습관 형성
- 기존 프로젝트 1개 모듈 @Codebase 분석
- Java: JUnit 5 커버리지 80% 목표 설정
- C++: Google Test + CMake CI 파이프라인 구축
-  Python: uv + pytest + ruff 환경 셋업

### 3개월

확장 — 팀 프로세스 통합

- PR 자동 AI 리뷰 워크플로우 운영 안정화
- 레거시 모듈 1~2개 AI 지원 리팩토링 완료
- 팀 Prompting BP 가이드 문서 v1.0 발행
- 커버리지 80%+ CI 게이트 전체 적용
-  Python: FastAPI 서비스 1개 AI 리팩토링
- 사내 AI 도구 활용 세미나 발표 1회

### 6개월

내재화 — 조직 문화 정착

- AI 코드 품질 지표 대시보드 구축
- 신규 팀원 온보딩: AI 활용 가이드 포함
- MCP 연동: GitHub Issues→코드→PR 자동화
-  Python: ML 파이프라인 AI 지원 도입
- 팀 AI 활용 성과 측정 (개발 시간 단축 정량화)
- 사내 Best Practice 발표 & 타 팀 전파

 최종 제출 회고 보고서: # Prompting BP 회고 ## 1.Keep ## 2.Problem ## 3.Try ## 4.개인별 학습 ## 5.팀 액션 아이템 ## 6.현업 적용 계획

 Prompting\_BP\_xx 레포지토리에 report.md 커밋 + GitHub PR 생성 + 리뷰어 등록

10분

 준비 & 체크인

- ESVP 체크 (Explorer·Shopper·Vacationer·Prisoner)
- 실습 3주 여정 감정 온도계 (● 좋음 ● 보통 ● 어려움)
- 시간축: 1주차(TDD/리팩토링)→2주차(Cursor AI)→3주차(팀)

10분

 데이터 수집

- 4L 매트릭스 (포스트잇·개인 작성→벽 게시)
- Learned·Lacked·Liked·Longed for
- Java/C++/Python 각 언어별 경험 색깔 구분 (🍌/🍌/🍌)

10분

 통찰 도출

- 유사 항목 군집화 → 브레인스토밍
- 점투표로 우선순위 설정 (각자 3개 점스티커)
- 생선가시 다이어그램: 왜 Cursor AI 활용이 효과적이었나?

10분

 SMART 액션 플랜

- 팀 공용 .cursorrules 작성 계획 → 책임자·기한 배정
- 개인별 3개월 학습 계획 카드 작성 (현업 적용 포함)
- 다음 실습 개선 규칙 3가지 선정 (Python 포함)

20분

 발표 & 마무리

- 팀별 6~7분 발표: Keep 2+Problem 2+Try 2
- BP 공유: C++·Java·Python 각 1개씩 실전 시연
- 감사 피드백 + 다음 과정(Gilded Rose) 연결

# 좋은 프롬프트는 좋은 질문에서 시작한다. 코드도, 시도, 성장도.



✓ Keep: .cursorrules에 Best Practice를 등록하면 팀 전체의 AI가 개선된다



✓ Problem: 모호한 프롬프트는 P-C-T-F + 실행 검증 요청으로 해결할 수 있다



✓ Try: 단계적 현업 적용 — 1개월(도구 정착) → 3개월(팀 통합) → 6개월(문화 내재화)



✓ C++ & Java: 언어·버전·빌드도구 명시 프롬프트가 가장 효과적이다



✓ Python: 타입 힌트·패키지 관리자(uv)·테스트 프레임워크(pytest) 명시 필수



C++17



Java 21



Python 3.12

수고하셨습니다 🎉